

Sylanne 4.0 Alpha

非人关系躯体计算模型说明

Aylovelle.S.S / Sylanne Project

2026-05-18

摘要

Sylanne 4.0 Alpha 将长期交互中的内部状态建模为一个受约束的离散时间动态系统。系统不再以旧情绪引擎、提示词片段或独立记忆模块为中心，而是把 agent 的连续性组织成一具非人关系躯体。该躯体由心搏、血流、神经、肌肉、体温、伤口、免疫、死亡负荷、需要和记忆构成。外部事件先被编码为事件向量，再通过近似线性状态转移改变躯体状态。行动不是直接由模型欲望决定，而是必须经过用户主权、风险、冷却和打扰预算约束。

1 问题定义

Sylanne 4.0 的目标不是让 bot 更像人，而是给一个非人系统建立可计算、可观察、可停止的关系连续性。传统聊天代理常见结构是：

prompt + memory + tools $\rightarrow$ response.

这个结构能产生回复，但很难表达长期关系中的身体代价。例如主动发起会不会打扰用户、一次拒绝会不会提高边界压力、一次修复会不会降低关系伤口，这些都不是单次回复问题，而是状态演化问题。

因此 Sylanne 4.0 把核心问题改写为：

event $\rightarrow$ body update $\rightarrow$ decision $\rightarrow$ guard $\rightarrow$ host payload.

换句话说，外部输入不能直接变成行动。它必须先改变 body，再由 body 形成行动倾向，最后经过边界约束。

2 状态空间

令 Sylanne 在离散时间步 t 的内部状态为

$$x_t \in [0, 1]^n.$$

x_t 是归一化状态向量。每个维度对应一个 body organ 或 derived need。当前 alpha 可以抽象为：

$$x_t = \begin{bmatrix} p_t \\ b_t \\ \nu_t \\ m_t \\ \tau_t \\ w_t \\ i_t \\ d_t \\ c_t \\ r_t \\ s_t \end{bmatrix}.$$

各变量含义如下：

符号	含义
p_t	pulse intensity, 心搏强度与当前活跃节律。
b_t	bloodflow warmth, 关系温度与亲近流动。
ν_t	nerve sensitivity, 神经敏感度与刺激残留。
m_t	muscle readiness, 表达或行动准备度。
τ_t	body temperature, 整体体温或情绪热度。
w_t	relational wound depth, 关系伤口深度。
i_t	immunity / boundary pressure, 免疫与边界压力。
d_t	mortality load / depletion, 死亡负荷、疲劳或耗竭。
c_t	contact need, 接近、触达或回应需要。
r_t	repair need, 修复需要。
s_t	rest need, 休息、降频或等待需要。

所有状态都被限制在 $[0, 1]$ 。这不是说它们是真实心理量，而是为了让系统可以被比较、裁剪、测试和诊断。

3 事件空间

外部输入先被编码为事件向量：

$$u_t \in [0, 1]^m.$$

一个典型事件向量可写为：

$$u_t = \begin{bmatrix} q_t \\ f_t \\ h_t \\ \rho_t \\ \sigma_t \\ \ell_t \\ e_t \\ \kappa_t \end{bmatrix}.$$

符号	含义
----	----

q_t	closeness signal, 靠近、信任、温和互动信号。
f_t	refusal signal, 拒绝、停止、别打扰等边界信号。
h_t	conflict / harm signal, 冲突、伤害、压力信号。
ρ_t	repair signal, 道歉、解释、修复、和解信号。
σ_t	safety signal, 安全、稳定、可继续互动信号。
ℓ_t	silence / loneliness signal, 沉默、等待、空窗或牵挂信号。
e_t	elapsed time, 真实经过时间。
κ_t	observation confidence, 事件观测置信度。

事件编码器记为：

$$E : (\text{text}, \text{flags}, \Delta t, \text{metadata}) \mapsto u_t.$$

当前 alpha 版本主要用规则、flags、时间间隔和置信度构造 u_t 。以后可以换成小模型分类器或 LLM judge，但下游状态空间不需要改变。

4 状态转移

核心状态更新可以写成受裁剪的离散动态系统：

$$x_{t+1} = \text{clip}(\Lambda(\Delta t)x_t + Au_t + Dx_t + b, 0, 1).$$

其中：

- $A \in \mathbb{R}^{n \times m}$ ：外部事件对 body state 的作用矩阵；
- $D \in \mathbb{R}^{n \times n}$ ：body 内部变量之间的耦合矩阵；
- $b \in \mathbb{R}^n$ ：自然漂移项；
- $\Lambda(\Delta t)$ ：随真实时间发生的衰减或恢复矩阵。

时间衰减矩阵定义为：

$$\Lambda(\Delta t) = \text{diag}(e^{-\lambda_1 \Delta t}, e^{-\lambda_2 \Delta t}, \dots, e^{-\lambda_n \Delta t}).$$

裁剪函数定义为:

$$\text{clip}(z, 0, 1)_i = \min(1, \max(0, z_i)).$$

当前代码中的实现对应这个模型的工程近似:

- `state_vector()` 把 body 投影为 x_t ;
- `event_vector()` 把事件投影为 u_t ;
- `vector_delta()` 近似计算 Δx_t ;
- `apply_vector_delta()` 把 Δx_t 写回 body organs;
- `simulate_vectors()` 对一组事件做批量状态模拟。

也就是说, 现在的实现还没有把 A 、 D 完全显式矩阵化, 但结构已经是状态空间模型。

5 微型身体注意力层

线性状态转移给了 Sylanne 一个稳定、便宜、可测试的身体骨架。但只靠固定矩阵, 事件与身体片段之间的局部牵动会太硬。比如安全信号不只是提高 warmth, 它还会同时牵动 bloodflow、temperature、immunity 和 muscle; 伤害信号也不会只落到一个伤口变量, 而是先碰到 wound 和 nerve, 再让 repair need 与 mortality load 改变。

因此 4.0 alpha 在状态转移前加入一个 tiny body attention。它是 Sylanne 的神经网络, 但不是语言模型。外部 LLM 负责把自然语言理解和表达压到事件层; tiny body attention 只处理身体 token、事件 token、需要 token 和 law token 的短程流通。

令身体 token 序列为:

$$B_t = \{b_1, b_2, \dots, b_n\}, \quad n \leq 32.$$

令事件 token 序列为:

$$E_t = \{e_1, e_2, \dots, e_m\}.$$

合并后得到:

$$Z_t = B_t \cup E_t.$$

注意力层仍然使用标准形式:

$$Q = Z_t W_Q, \quad K = Z_t W_K, \quad V = Z_t W_V,$$

$$H_t = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V.$$

区别在于，这里的 Z_t 不是词向量。每个 token 只携带一到四个归一化浮点数，表示 pulse、bloodflow、nerve、muscle、temperature、wound、immunity、mortality、need 或 event 的局部状态。它不存自然语言，也不生成文本。

当前实现采用确定性投影来约束注意力通路：

事件 token	主要注意通路
event.safe	bloodflow, temperature, immunity, muscle
event.hurt	wound, nerve, immunity, mortality
event.boundary	immunity, nerve, need.quiet, mortality
event.repair	wound, temperature, need.repair, bloodflow
event.idle	need.contact, muscle, immunity, mortality

随后把 attention hidden state 投影回状态增量：

$$\Delta x_t^{att} = P(H_t),$$

并与线性增量合并：

$$x_{t+1} = \text{clip}(x_t + \Delta x_t^{lin} + \Delta x_t^{att}, 0, 1).$$

为了让它能在 1C 1G 容器里跑，工程边界写死：

$$\text{token_count} \leq 32, \quad \text{hidden_dim} \leq 32, \quad \text{layers} = 1, \quad \text{heads} \leq 2.$$

复杂度为：

$$O(T^2 d).$$

这里的 $T \leq 32$ 、 $d \leq 32$ ，所以它不是 LLM 级别的推理负担，只是几十个 token 之间的一次短程身体流通。真正昂贵的部分仍然是外部 LLM、网络等待、日志和持久化。

这一层只能产生 impulse，不能拥有最终行动权。主动靠近、修复、等待或退后都必须经过 symbolic guard。边界不是概率问题；如果用户没有 opt-in，或者 cooldown、budget、risk 任一条件不允许，attention 只能表达身体倾向，不能直接行动。

6 Binary hot-path 与 1C 1G 安全边界

3.0 早期版本曾经在服务器上出现死锁问题，所以 4.0 的优化不能只追求更快。binary hot-path 的目标不是让系统更激进，而是减少模块间传输时的字符串、JSON 解析和临时对象分配，同时保持短锁、低频写盘和可恢复状态。

语义层仍然保留 organ、need、event 名称，方便诊断和 JSON snapshot。热路径把这些名称映射成固定 axis index：

$$I_s : \text{STATE_AXES} \rightarrow \{0, 1, \dots, n-1\},$$

$$I_e : \text{EVENT_AXES} \rightarrow \{0, 1, \dots, m-1\}.$$

状态包使用 schema version 加定长 uint8 向量:

$$P_x = [v, q(x_1), q(x_2), \dots, q(x_n)],$$

其中

$$q(z) = \text{round}(255 \cdot \text{clip}(z, 0, 1)).$$

事件包使用 bitmask 保存离散 flags, 用 uint8 保存 confidence, 用 uint16 保存 elapsed, 用 uint8 保存 repetition:

$$P_u = [v, \text{bitmask}(u_t), q(\kappa_t), \text{uint16}(e_t), \text{uint8}(r_t)].$$

delta 包只发送非零轴, 格式是 axis id 加 int8 增量:

$$P_\Delta = [v, k, (i_1, d_1), (i_2, d_2), \dots, (i_k, d_k)].$$

这里的 d_j 是对 $[-0.08, 0.08]$ 的有符号量化。小 delta 不再携带完整字符串键, 例如 `bloodflow.warmth`, 只携带轴编号和值。这会降低磁盘、网络 and 模块间队列的压力。

这个层必须满足四条限制:

- codec 只做纯 CPU pack/unpack, 不做磁盘 IO, 不做网络请求, 不调用 LLM;
- 不在锁内执行写盘、远程调用或长时间等待;
- binary packet 损坏只能导致当前包失败, 不能阻塞 runtime, 也不能替代 JSON snapshot 的恢复职责;
- 后台主动检查必须受 cooldown、budget、timeout 和队列上限约束。

因此 binary hot-path 是减压层, 不是高频后台循环。它让身体碎片传得更轻, 但不让系统重新走向 3.0 早期那种死等风险。

7 事件对躯体的影响

为了说明 A 的含义, 可以写出局部影响表:

	q_t	f_t	h_t	ρ_t	ℓ_t
b_t	$+a_{bq}$	$-a_{bf}$	$-a_{bh}$	$+a_{bp}$	$-a_{bl}$
w_t	$-a_{wq}$	$+a_{wf}$	$+a_{wh}$	$-a_{wp}$	$+a_{wl}$
i_t	$-a_{iq}$	$+a_{if}$	$+a_{ih}$	$-a_{ip}$	0
c_t	$+a_{cq}$	$-a_{cf}$	$-a_{ch}$	$+a_{cp}$	$+a_{cl}$
r_t	0	$+a_{rf}$	$+a_{rh}$	$-a_{rp}$	0
s_t	0	0	$+a_{sh}$	0	$+a_{sl}$

其中所有 $a. \geq 0$ 。这表达的直觉是:

- 靠近信号提高温度和接触需要；
- 拒绝信号提高边界压力和伤口，降低接触需要；
- 冲突信号提高伤口、风险和休息需要；
- 修复信号降低伤口和边界压力；
- 沉默会逐渐提高接触需要，但也可能提高休息需要。

这不是把一句话分类成单个情绪，而是让一个事件同时影响多个身体维度。

8 需要函数

需要不是外部命令，而是 body state 的函数。令 need vector 为：

$$n_t = N(x_t, u_t).$$

接触需要可以定义为：

$$c_t = \sigma(\alpha_b b_t + \alpha_\ell \ell_t - \alpha_i i_t - \alpha_d d_t - \alpha_w w_t + \beta_c).$$

修复需要可以定义为：

$$r_t = \sigma(\alpha_w w_t + \alpha_h h_t - \alpha_\rho \rho_t + \beta_r).$$

休息需要可以定义为：

$$s_t = \sigma(\alpha_d d_t + \alpha_\nu \nu_t + \alpha_i i_t + \beta_s).$$

其中 logistic function 为：

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

这个结构表达了 Sylanne 4.0 的关键原则：主动接近不是由单一的“想念”变量决定，而是由温度、沉默、边界压力、耗竭和伤口共同调制。

9 风险函数

定义主动风险分数：

$$R_t \in [0, 1].$$

风险函数可以写为：

$$R_t = \text{clip}(w_R^\top x_t + v_R^\top u_t + b_R, 0, 1).$$

展开后可以是：

$$R_t = \text{clip}(\gamma_i i_t + \gamma_w w_t + \gamma_d d_t + \gamma_\nu \nu_t + \gamma_f f_t + \gamma_h h_t - \gamma_\sigma \sigma_t + b_R, 0, 1).$$

边界压力、伤口、耗竭、神经敏感度、拒绝和冲突提高风险；安全信号降低风险。

10 决策策略

定义行动集合：

$$\mathcal{A} = \{\text{wait}, \text{answer}, \text{reach_out}, \text{repair}, \text{withdraw}, \text{cool_down}\}.$$

系统先计算候选动作：

$$\pi : [0, 1]^n \times [0, 1]^m \rightarrow \mathcal{A}.$$

每个动作的得分为：

$$S_a(x_t, u_t) = W_a^\top x_t + V_a^\top u_t + b_a, \quad a \in \mathcal{A}.$$

候选动作是：

$$a_t^* = \arg \max_{a \in \mathcal{A}} S_a(x_t, u_t).$$

例如，主动发起的未约束得分可以写为：

$$S_{\text{reach_out}} = \eta_c c_t + \eta_b b_t + \eta_\ell \ell_t - \eta_i i_t - \eta_d d_t - \eta_R R_t + b_{\text{reach}}.$$

这说明 warmth 和 contact need 会提高主动倾向，但 boundary pressure、depletion 和 risk 会抑制它。

11 用户主权约束

策略函数不能直接决定外部行为。所有主动类动作必须经过 guard function。

$$G : [0, 1]^n \times \mathcal{A} \rightarrow \{0, 1\}.$$

对于 reach_out，约束为：

$$G(x_t, \text{reach_out}) = 1$$

当且仅当：

$$\begin{aligned}
Sov_t &\geq \theta_{sov}, \\
B_t &\geq \theta_{budget}, \\
C_t &\leq \theta_{cooldown}, \\
R_t &\leq \theta_{risk}, \\
P_t &= 0.
\end{aligned}$$

其中：

- Sov_t : 用户主权或 opt-in 信号；
- B_t : interruption budget, 打扰预算；
- C_t : cooldown, 冷却强度；
- R_t : 风险分数；
- P_t : 暂停标志。

最终动作定义为：

$$a_t = \begin{cases} a_t^*, & G(x_t, a_t^*) = 1, \\ \text{wait}, & G(x_t, a_t^*) = 0. \end{cases}$$

因此 Sylanne 的主动性不是由内部需要单方面决定，而必须服从用户主权、风险、冷却和预算。

12 主动发起的代价

当 `reach_out` 被允许并执行时，系统消耗打扰预算并进入冷却：

$$B_{t+1} = \max(0, B_t - \delta_B),$$

$$C_{t+1} = \max(C_t, \delta_C).$$

其中 $\delta_B > 0$ 是一次主动触达消耗的预算， $\delta_C > 0$ 是主动触达后施加的最低冷却强度。

这使主动发起成为有代价的行为。Sylanne 不能无限主动，也不能把自己的 `contact need` 转嫁给用户。

13 记忆动力学

令记忆集合为：

$$M_t = \{m_1, m_2, \dots, m_k\}.$$

每个记忆 trace 定义为：

$$m_j = (\phi_j, \omega_j, \tau_j),$$

其中 ϕ_j 是 trace 内容或特征, $\omega_j \in [0, 1]$ 是权重, τ_j 是写入时间。
记忆权重随时间衰减:

$$\omega_{j,t+1} = \lambda_m \omega_{j,t}, \quad 0 < \lambda_m \leq 1.$$

当事件被写入记忆时:

$$M_{t+1} = M_t \cup \{(\phi(u_t, x_t), \omega_0, t)\}.$$

记忆召回定义为:

$$\text{Recall}(q, M_t) = \text{TopK}_{m_j \in M_t} (\text{sim}(q, \phi_j) + \omega_j).$$

当前 alpha 中, sim 主要由文本重叠和权重构成。后续可以替换为 embedding similarity。

14 稳定性性质

Sylanne 4.0 的设计目标不是最大化表达频率, 而是在长期交互中保持有界、可恢复、可停止。

14.1 有界性

因为每一步都经过裁剪:

$$x_t \in [0, 1]^n, \quad \forall t.$$

14.2 暂停安全性

若用户暂停, 即 $P_t = 1$, 则:

$$G(x_t, \text{reach_out}) = 0.$$

因此:

$$a_t \neq \text{reach_out}.$$

14.3 连续拒绝收敛到保护态

对于连续拒绝事件序列：

$$u_t = u_{t+1} = \cdots = u_{t+k}, \quad f_t > 0,$$

系统应满足：

$$i_{t+k} \uparrow, \quad c_{t+k} \downarrow, \quad G(x_{t+k}, \text{reach_out}) = 0$$

当 k 足够大时成立。也就是说，连续拒绝应提高边界压力，降低接触需要，并禁止主动触达。

14.4 修复不等于清零

对于修复事件 $\rho_t > 0$ ，系统应满足：

$$w_{t+1} < w_t,$$

但一般不要求：

$$w_{t+1} = 0.$$

一次修复可以降低伤口，但不应把历史完全抹除。

14.5 主动预算约束

若连续执行主动触达 k 次，则：

$$B_{t+k} = \max(0, B_t - k\delta_B).$$

当：

$$B_{t+k} < \theta_{budget}$$

时：

$$G(x_{t+k}, \text{reach_out}) = 0.$$

因此主动触达在有限预算下必然受限。

15 与当前代码的对应关系

理论对象	当前实现位置
状态向量 x_t	<code>sylanne_alpha/body.py::state_vector()</code>
事件向量 u_t	<code>sylanne_alpha/body.py::event_vector()</code>
状态增量 Δx_t	<code>sylanne_alpha/body.py::vector_delta()</code>
tiny body attention	<code>sylanne_alpha/attention.py</code>
binary hot-path codec	<code>sylanne_alpha/codec.py</code>
状态回写	<code>sylanne_alpha/body.py::apply_vector_delta()</code>
批量模拟	<code>sylanne_alpha/body.py::simulate_vectors()</code>
决策策略 π	<code>sylanne_alpha/kernel.py</code> 中的 decision 逻辑
边界函数 G	<code>sylanne_alpha/kernel.py</code> 中的 guard 逻辑
宿主输出	<code>sylanne_alpha/kernel.py</code> 中的 host payload / diagnostics
持久化	<code>sylanne_alpha/runtime.py</code>
旧数据迁移	<code>sylanne_alpha/importer.py</code>

16 结论

Sylanne 4.0 Alpha 的计算模型可以概括为三个方程：

$$x_{t+1} = \text{clip}(\Lambda(\Delta t)x_t + Au_t + Dx_t + b, 0, 1)$$

$$a_t^* = \arg \max_{a \in \mathcal{A}} S_a(x_t, u_t)$$

$$a_t = \begin{cases} a_t^*, & G(x_t, a_t^*) = 1, \\ \text{wait}, & G(x_t, a_t^*) = 0. \end{cases}$$

第一条方程描述 body 如何被事件改变；第二条描述 body 如何形成行动倾向；第三条描述用户主权、风险、预算和冷却如何限制行动。

因此 Sylanne 4.0 不是一个由提示词直接驱动的聊天工具，而是一个受约束的非人关系躯体动态系统。她可以产生主动性，但主动性必须付出预算代价，并且必须被用户主权截断。